

PRODUCT REQUIREMENTS DOCUMENT

Final Project

Multi-Vendor & Single-Vendor E-Commerce Platform
Built on the MERN Stack

Version	1.0
Date	19th February 2026
Status	APPROVED FOR DEVELOPEMENT
Type	Final Project
Deadline	12th March 2026

1. Executive Summary

Final Project is a MERN-stack e-commerce platform designed as a Project/student capstone project. It solves a real problem for small-scale local sellers, artisans, boutique owners, and independent creators, who lack affordable, easy-to-use digital storefronts. Existing marketplaces are either too complex to set up, charge excessive commissions, or do not serve niche product communities.

1.1 Platform Modes

Final Project ships in two distinct modes which the student chooses at project init time:

Mode	Description	Best For
Multi-Vendor Marketplace	Multiple sellers register, list products, and customers buy from any vendor in one cart. Admin manages platform and takes commission.	General-purpose marketplace, Project judges expect multi-actor systems.
Single-Vendor Store	One brand/seller owns the store. Admin = Vendor. Customers browse & buy a curated product catalogue. Commission logic disabled.	Niche product stores: e.g. handmade jewellery, streetwear, artisanal food, limited-edition prints.

Design Decision, Choosing a Mode

Both modes share the same codebase. A single environment variable (STORE_MODE = 'multi' | 'single') at the backend and a feature flag on the frontend toggles vendor-registration UI, commission dashboards, and multi-vendor cart logic. This means a student can build once and demo either mode without a fork.

2. Problem Statement & Solution

2.1 The Problem

- Local artisans and boutique owners have no affordable digital presence.
- Setting up Shopify costs money; building a custom site requires technical expertise.
- Large marketplaces (Amazon, Daraz) take 15–30% commissions and bury small sellers.
- Existing open-source solutions are too complex for a Project timeframe.
- Niche sellers (e.g., handmade soap brand) need a single-brand store, not a big marketplace.

2.2 The Solution

A lightweight, MERN-powered platform with two operating modes from which the student chooses one:

- Multi-Vendor: Any verified user can become a vendor, list products, and receive payouts minus platform commission.
- Single-Vendor: One brand control the store, ideal for a student who wants to launch their own niche e-commerce brand as a project.
- Customers always experience a seamless single-checkout flow regardless of how many sellers are in the cart.

3. Goals, Objectives & Success Metrics

3.1 Primary Goals

1. Deliver a working MERN e-commerce app within 3 weeks Project window.
2. Support both multi-vendor and single-vendor modes via config.
3. Demonstrate JWT authentication, role-based access, REST APIs, and third-party integrations (Cloudinary + Stripe).
4. Maintain clean, documented code that instructors can review.

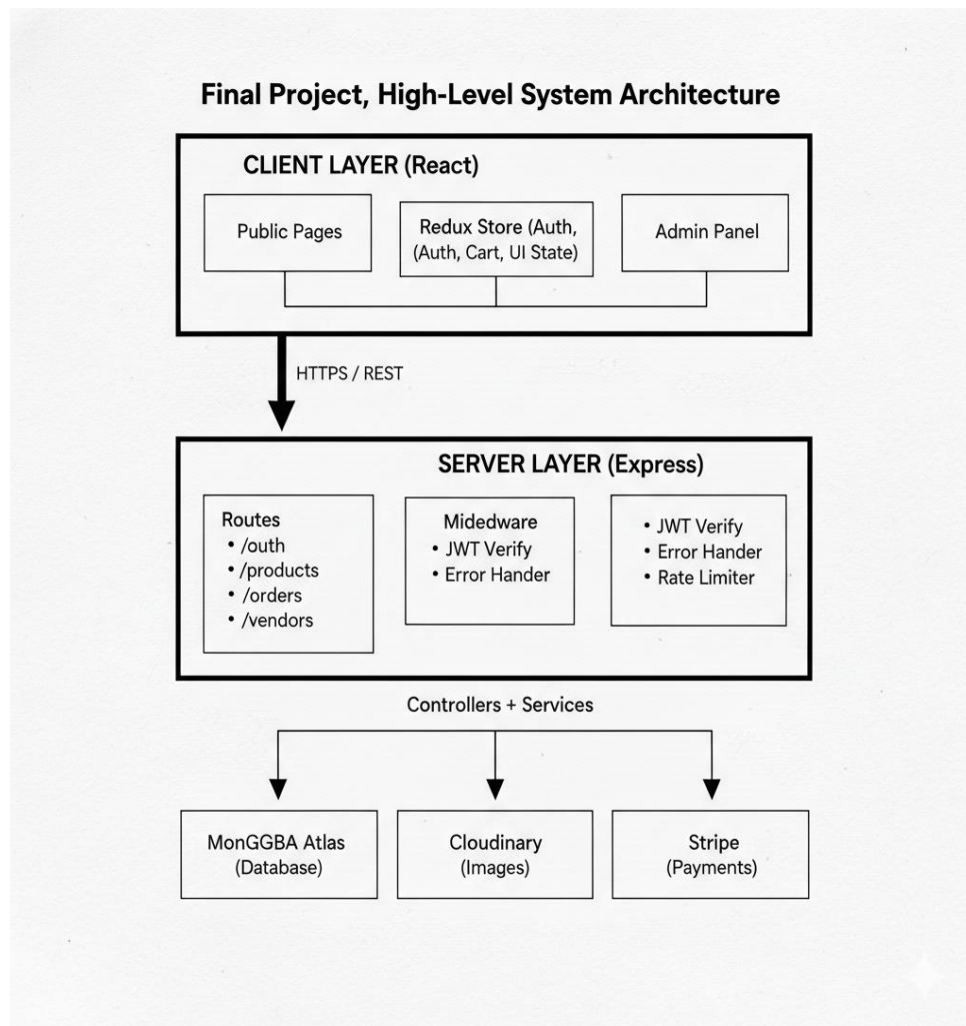
3.2 Success Metrics

Metric	Target (MVP)	Stretch Goal
Vendor onboarding time	< 3 minutes	< 1 minute
Product listing time	< 2 minutes	< 30 seconds
Checkout steps	<= 3 steps	2 steps (cart → Stripe)
API response time	< 500ms	< 200ms
Role-based route protection	100% routes protected	100% + rate limiting
Mobile responsiveness	Basic responsive	Full mobile-first
Stripe test payment success	>= 95% test transactions	100%

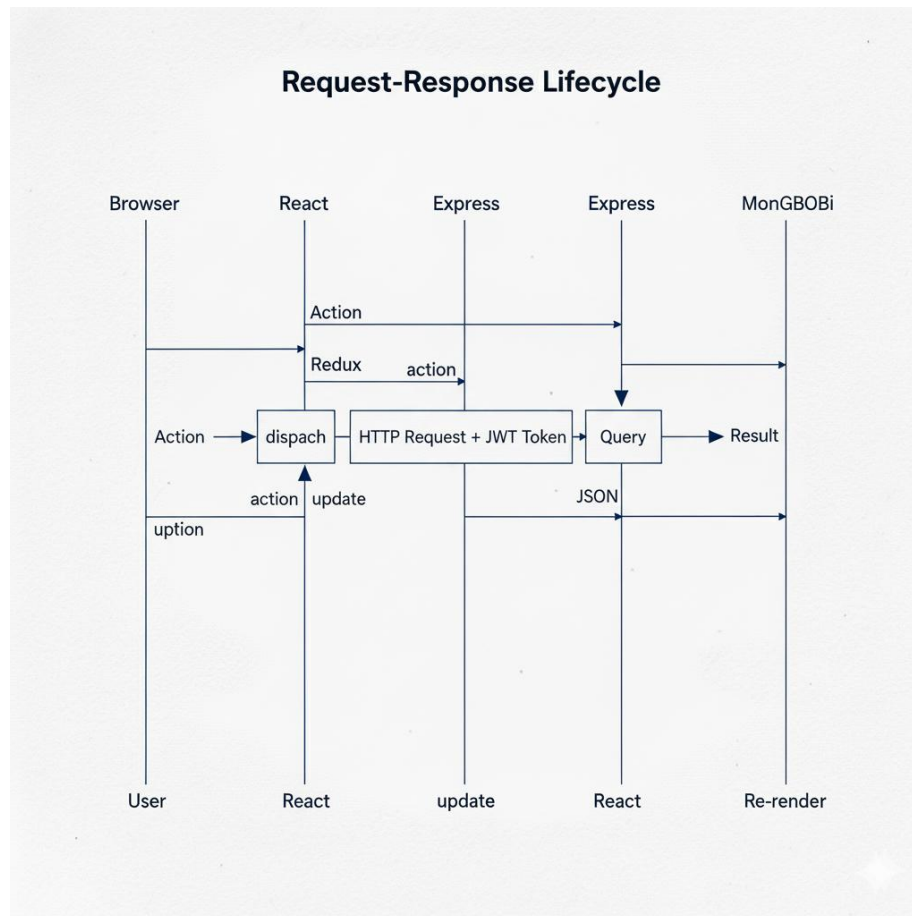
4. System Architecture & Design

4.1 High-Level Architecture Diagram

The following diagram illustrates the overall system architecture of Final Project:



4.2 Request-Response Lifecycle



4.3 Directory Structure

Project Folder Structure	
Final Project/	
├─ client/	# React Frontend
├─ public/	
└─ src/	
├─ api/	# Axios instance + API calls
├─ app/	# Redux store setup
├─ components/	# Reusable UI components
├─ common/	# Button, Input, Modal, Spinner
├─ layout/	# Navbar, Footer, Sidebar
└─ product/	# ProductCard, ProductGrid
├─ features/	# Redux slices
├─ authSlice.js	
├─ cartSlice.js	
└─ productSlice.js	
├─ pages/	# Route-level page components
├─ auth/	# Login, Register
├─ customer/	# Home, Product, Cart, Orders
├─ vendor/	# Dashboard, AddProduct, Sales
└─ admin/	# AdminDashboard, UserMgmt
├─ routes/	# ProtectedRoute, RoleRoute
├─ hooks/	# useAuth, useCart custom hooks
└─ utils/	# formatCurrency, validators
└─ server/	# Node.js + Express Backend
├─ config/	# db.js, cloudinary.js, stripe.js
├─ controllers/	# authController, productController
├─ middleware/	
├─ authMiddleware.js	# JWT verify
├─ roleMiddleware.js	# Admin / Vendor / Customer guard
└─ errorMiddleware.js	# Global error handler
├─ models/	# Mongoose schemas
├─ User.js	
├─ Product.js	
└─ Order.js	
├─ routes/	# Express routers
├─ authRoutes.js	
├─ productRoutes.js	
├─ orderRoutes.js	
└─ vendorRoutes.js	
├─ services/	# Business logic layer
├─ utils/	# generateToken, sendEmail
└─ index.js	# Entry point

5. Database Schema (ERD)

5.1 Entity Relationship Diagram



5.2 Mongoose Schema Details

User Schema

Field	Type	Required	Notes
_id	ObjectId	Auto	MongoDB default primary key
name	String	Yes	Min 2, Max 50 chars
email	String	Yes	Unique, lowercased, validated
password	String	Yes	bcrypt hashed, min 8 chars
role	Enum	Yes	'Admin' 'Vendor' 'Customer'
storeInfo.name	String	No	Single-vendor mode: store name
storeInfo.description	String	No	Vendor's store bio
storeInfo.logo	String	No	Cloudinary URL
isActive	Boolean	No	Default true; Admin can deactivate
createdAt	Date	Auto	Mongoose timestamps

Product Schema

Field	Type	Required	Notes
name	String	Yes	Max 120 chars
description	String	Yes	Max 2000 chars, supports Markdown
price	Number	Yes	Min 0, stored in smallest currency unit
images[]	String[]	Yes	Min 1 image, Cloudinary URLs
category	String	Yes	From predefined enum list
stock	Number	Yes	Default 0, decremented on order
vendorId	ObjectId ref	Yes	References User with Vendor role
isActive	Boolean	No	Soft delete flag

6. REST API Design

6.1 API Routes Overview

REST API Routes		
BASE URL: https://api.Final Project.com/api (or localhost:5000/api)		
AUTH ROUTES (/auth)		
└─ POST	/register	→ Register user (role in body)
└─ POST	/login	→ Returns JWT token
└─ GET	/profile	→ Get current user [JWT]
└─ PUT	/profile	→ Update name/password [JWT]
PRODUCT ROUTES (/products)		
└─ GET	/	→ All active products (public)
└─ GET	/:id	→ Single product (public)
└─ POST	/	→ Create product [Vendor/Admin]
└─ PUT	/:id	→ Update product [Owner Vendor]
└─ DELETE	/:id	→ Soft delete [Owner Vendor/Admin]
└─ GET	/vendor/mine	→ Vendor's own products [Vendor]
ORDER ROUTES (/orders)		
└─ POST	/	→ Create order + Stripe session [Customer]
└─ GET	/mine	→ Customer's orders [Customer]
└─ GET	/vendor/sales	→ Vendor's received orders [Vendor]
└─ GET	/	→ All orders [Admin]
└─ GET	/:id	→ Single order [Owner/Admin]
└─ PUT	/:id/status	→ Update order status [Vendor/Admin]
VENDOR ROUTES (/vendors)		
└─ GET	/	→ List all vendors [Admin/Public]
└─ GET	/:id	→ Vendor public profile (multi-vendor)
ADMIN ROUTES (/admin)		
└─ GET	/stats	→ Platform overview stats [Admin]
└─ GET	/users	→ All users [Admin]
└─ PUT	/users/:id	→ Toggle user active status [Admin]
└─ GET	/commission	→ Commission summary [Admin]
UPLOAD ROUTES (/upload)		
└─ POST	/	→ Get Cloudinary signed URL [Vendor]

```
STRIPE WEBHOOK (/stripe/webhook)
└─ POST    /                               → Handle payment confirmation
```

6.2 Sample Request / Response

POST /api/auth/login

Sample: Login Endpoint

Request:

```
{
  "email": "vendor@example.com",
  "password": "securepassword"
}
```

Response 200 OK:

```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "_id": "64a3f...",
    "name": "Ayesha Khan",
    "email": "vendor@example.com",
    "role": "Vendor"
  }
}
```

Response 401 Unauthorized:

```
{
  "success": false,
  "message": "Invalid credentials"
}
```

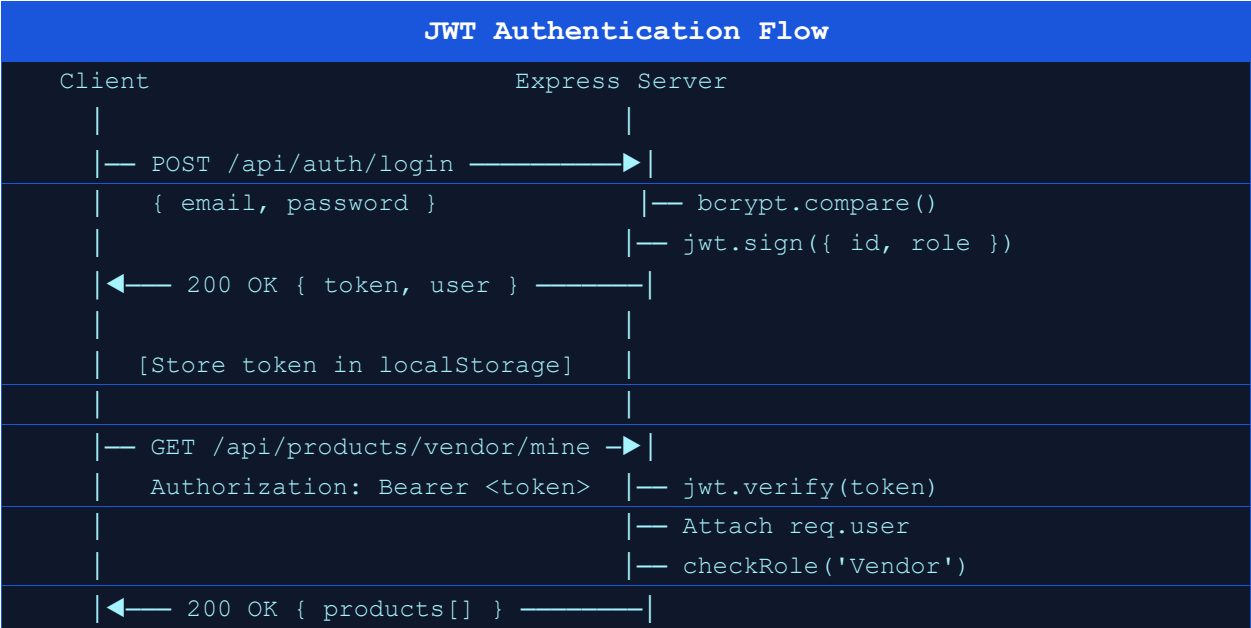
7. Functional Requirements

7.1 Authentication & Role System

User Roles & Permissions Matrix

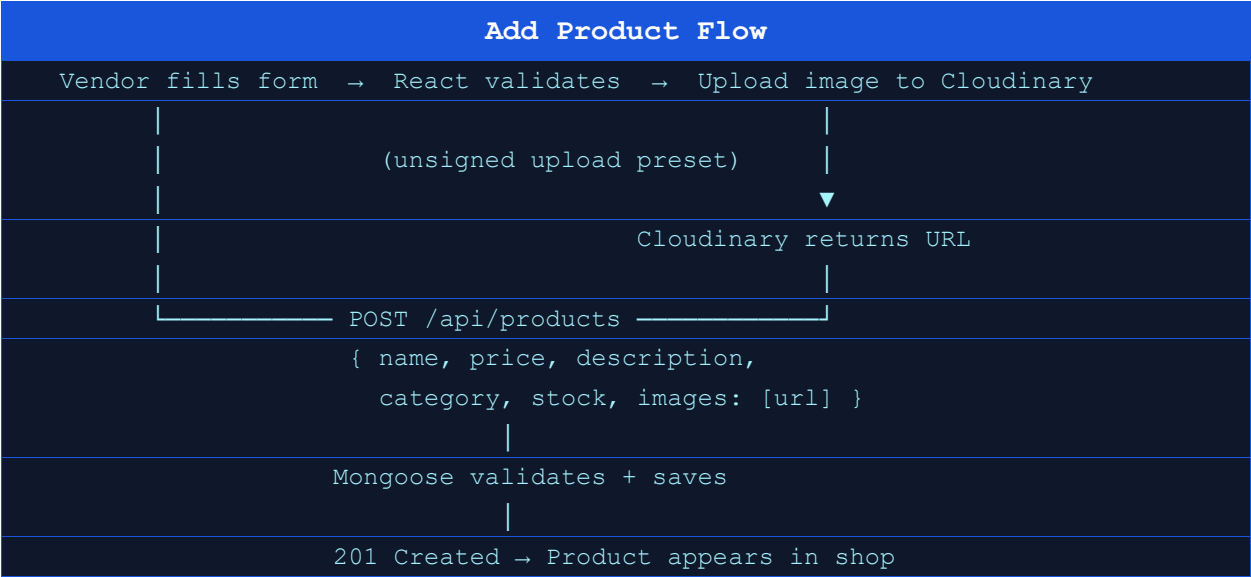
Capability	Guest	Customer	Vendor	Admin
Browse products	☒	☒	☒	☒
Register / Login	☒	☒	☒	☒
Add to cart	☐	☒	☒	☒
Checkout & Pay	☐	☒	☒	☒
View own orders	☐	☒	☒	☒
Add products	☐	☐	☒	☒
Edit own products	☐	☐	☒	☒
View sales dashboard	☐	☐	☒	☒
Update order status	☐	☐	☒	☒
Delete any product	☐	☐	☐	☒
Manage all users	☐	☐	☐	☒
View platform revenue	☐	☐	☐	☒

JWT Flow Diagram



7.2 Vendor Module

Add Product Flow



Vendor Dashboard Data

Widget	Data Source	Calculation
Total Revenue	Order.items where vendorId = mine	SUM(price * qty)

Widget	Data Source	Calculation
Platform Fee (10%)	Derived from revenue	revenue * 0.10
Net Earnings	Derived	revenue - platformFee
Orders This Month	Orders grouped by createdAt month	COUNT(orders)
Top Products	Order items sorted by qty	Aggregation pipeline
Stock Alerts	Products where stock <= 5	Mongoose query

7.3 Shopping Cart

Cart State Structure (Redux)

Redux Cart State Shape
<pre>cartState = { items: [{ productId: '64a3f...', name: 'Handmade Soap', price: 350, // in PKR / selected currency qty: 2, image: 'https://cloudinary.../soap.jpg', vendorId: '64b1e...', // stored for vendor payout split stock: 15, // to enforce stock limit in UI }], itemCount: 2, subtotal: 700, lastUpdated: '2026-02-10T12:00:00Z' }</pre>
Persistence: cartState is synced to localStorage on every Redux action. On app load, middleware rehydrates Redux store from localStorage.

7.4 Checkout & Stripe Integration



8. Single-Vendor vs. Multi-Vendor Mode Deep Dive

8.1 Feature Comparison

Feature	Single-Vendor	Multi-Vendor
Vendor Registration	Disabled (admin is only seller)	Open or invite-based
Product Ownership	All products belong to the store	Each product has vendorId
Cart Logic	Standard cart	Multi-vendor cart with vendorId per item
Commission Calculation	Not applicable	10% platform fee on each sale
Vendor Dashboard	Admin manages all products	Each vendor sees own dashboard
Seller Profiles	Single brand page	Individual vendor store pages
Order Routing	All orders to admin	Orders split by vendor for fulfillment
Environment Variable	STORE_MODE=single	STORE_MODE=multi
Use Case Example	Artisan soap brand, sneaker store	Local bazaar, campus marketplace

8.2 Single-Vendor Store Ideas for Students

Students choosing Single-Vendor mode should pick a niche product category for maximum authenticity. Here are recommended store concepts:

Store Concept	Product Category	Special Feature to Highlight
Handmade Craft Studio	Jewellery, home decor, pottery	Custom order request form
Artisanal Food Shop	Jams, pickles, baked goods	Expiry date field in product schema
Vintage Clothing Store	Pre-loved fashion items	Condition field: Good/Excellent/New
Print-on-Demand Store	T-shirts, mugs, posters	Size/variant selector in product page
Digital Products Store	eBooks, presets, templates	Delivery via email link (no shipping)
Plant & Seed Shop	Indoor plants, herb seeds	Care instructions field per product
Campus Merch Store	University hoodies, stationery	Bulk discount logic for quantities

9. Technical Stack & Technology Decisions

Layer	Technology	Version	Decision Rationale
Frontend UI	React	18.x	Industry standard, component model maps perfectly to e-commerce UI patterns
Styling	Tailwind CSS	3.x	Utility-first means fast Project styling without writing custom CSS files
State Mgmt	Redux Toolkit	2.x	Predictable state for cart is critical; RTK reduces boilerplate vs raw Redux
Routing	React Router DOM	6.x	Declarative routing with nested routes for protected areas
HTTP Client	Axios	1.x	Interceptors allow global JWT injection and error handling
Backend	Node.js + Express	20 LTS / 4.x	Same JavaScript language as frontend; enormous ecosystem
Database	MongoDB Atlas	7.x	Flexible schema makes adding product fields easy during Project
ODM	Mongoose	8.x	Schema validation, middleware hooks, and population simplify DB access
Auth	JWT + bcrypt	—	Stateless, scales well, bcrypt for password hashing
Image Storage	Cloudinary	—	Free tier, unsigned upload preset means no backend image proxy needed
Payments	Stripe	—	Pre-built Checkout page handles PCI compliance out of the box
Frontend Deploy	Vercel	—	Zero-config React deploy, instant CDN, free for students
Backend Deploy	Render	—	Free tier Node.js web service, auto-deploy from GitHub
Environment Config	dotenv	16.x	Manages secrets; STORE_MODE flag controls platform mode

10. Pages, Screens & UI Wireframe Descriptions

10.1 Page Inventory

Page / Route	Access	Mode	Key Components
/ (Home)	Public	Both	Hero banner, Featured products grid, Category filter bar
/products	Public	Both	Product Grid with search, filter by category/price, pagination
/products/:id	Public	Both	Product detail, image gallery, Add to Cart, vendor info
/cart	Customer	Both	Cart items, qty controls, subtotal, Checkout button
/checkout	Customer	Both	Shipping address form → redirect to Stripe
/order-success	Customer	Both	Order confirmation + order ID
/my-orders	Customer	Both	List of past orders with status badges
/auth/login	Guest	Both	Email + password form, link to register
/auth/register	Guest	Both	Name, email, password, role dropdown (multi: Buyer/Seller)
/vendor/dashboard	Vendor	Both	Stats cards, recent orders, stock alerts
/vendor/products	Vendor	Both	Product table with edit/delete actions
/vendor/products/add	Vendor	Both	Add product form + Cloudinary upload
/vendor/sales	Vendor	Both	Sales history, revenue breakdown, commission display
/admin/dashboard	Admin	Both	Platform stats: users, orders, revenue, commission
/admin/users	Admin	Multi	User table with role, active status toggle
/vendors/:id	Public	Multi	Vendor public store page with their products

10.2 Navigation Structure

Navigation Structure
Navbar (all users)
└ Logo / Store Name
└ Search Bar
└ Category Links
└ Cart Icon (with badge count)
└ Auth Section:
└ [Not logged in] → Login Register
└ [Customer] → My Orders Profile Logout
└ [Vendor] → Dashboard My Products Sales Logout
└ [Admin] → Admin Panel Logout
Sidebar (Vendor Dashboard only)
└ Overview
└ My Products
└ Add Product
└ Sales History
└ Account Settings

11. Implementation Roadmap (Project Timeline)

Phase 1, The Skeleton

Task	Owner	Acceptance Criteria
Init Git repo (monorepo)		client/ and server/ folders committed
Create Express server + /health route	Backend Work	GET /api/health returns 200 {ok: true}
Connect MongoDB Atlas	Backend Work	mongoose.connect() succeeds, logged
Create User, Product, Order models	Backend Work	Schemas created with all fields
Bootstrap React with Vite + Tailwind	Frontend Work	App renders with Tailwind classes
Setup react-router-dom	Frontend Work	/, /auth/login, /auth/register routes exist
Setup Redux store with cart + auth slices	Frontend Work	Redux WorkTools shows initial state
Setup Axios instance with JWT interceptor	Frontend Work	Interceptor adds Authorization header
Setup dotenv + STORE_MODE config	Backend Work	Both modes toggle without code change

Phase 2, The Engines

Task	Owner	Acceptance Criteria
POST /auth/register and /auth/login	Backend Work	Returns JWT token, bcrypt password stored
authMiddleware + roleMiddleware	Backend Work	Protected route rejects invalid tokens
Register & Login React pages	Frontend Work	User can register, login, token stored
ProtectedRoute component	Frontend Work	Unauthenticated redirect to /auth/login
CRUD product API routes	Backend Work	All 5 endpoints working with Postman

Task	Owner	Acceptance Criteria
Cloudinary unsigned upload integration	Full Stack	Image URL stored in product.images[]
Vendor Add Product page	Frontend Work	Form submits, product appears in DB
Product Grid (Home + /products page)	Frontend Work	Products load from API, displayed in grid
Product Detail page	Frontend Work	Single product displayed, Add to Cart works

Phase 3, The Flow

Task	Owner	Acceptance Criteria
Cart page with Redux + localStorage sync	Frontend Work	Cart persists on page refresh
POST /api/orders + Stripe session creation	Backend Work	Stripe checkout URL returned
Checkout page → Stripe redirect	Frontend Work	Customer lands on Stripe hosted page
Stripe webhook → update order status	Backend Work	Order.status = 'Paid' after payment
Order success page	Frontend Work	Confirmation shown post-payment
My Orders page (customer)	Frontend Work	Order list with status badges shown
Vendor Dashboard stats	Full Stack	Revenue, orders, top products displayed
Vendor Sales History page	Full Stack	Itemised sales with commission breakdown

Phase 4, Polish & Deploy

Task	Owner	Acceptance Criteria
Admin dashboard + user management	Backend + Frontend	Admin can deactivate users/vendors
Search + filter on product listing	Frontend Work	Results update in real-time

Task	Owner	Acceptance Criteria
Mobile responsiveness audit	Frontend Work	All key pages pass 375px viewport test
Error handling + loading states	Frontend Work	Spinners, empty states, toast notifications
Deploy backend to Render	Backend Work	API live at render.com URL
Deploy frontend to Vercel	Frontend Work	React app live at Vercel, app URL
Configure env vars on both platforms	Team Lead	No hardcoded secrets in codebase
End-to-end demo walkthrough	Full Team	Complete buyer and vendor journey works

12. Non-Functional Requirements

Category	Requirement	Implementation Approach
Security	All passwords hashed with bcrypt (salt rounds ≥ 10)	bcrypt.hash(password, 10) in User pre-save hook
Security	JWT expires in 24h; rotate on suspicious activity	jwt.sign with expiresIn: '24h'
Security	Vendor can only modify own products (ownership check)	Middleware compares product.vendorId with req.user._id
Security	No API key or secret stored in frontend codebase	Cloudinary unsigned preset; Stripe keys backend-only
Performance	Product listing API response $< 500\text{ms}$	Index on category, vendorId, isActive fields
Performance	Images served via Cloudinary CDN	URLs stored in DB, never served through Express
Reliability	MongoDB Atlas free tier: 512MB limit awareness	Limit image uploads, use soft deletes not hard
Scalability	Stateless Express server (JWT, no sessions)	Allows horizontal scaling; Render auto-scales
Usability	Onboard a vendor in under 3 minutes	Minimal form fields; inline validation; pre-filled examples
Maintainability	No business logic in route files	Controllers handle logic; services handle DB queries
Observability	Request logging in Development	Morgan middleware: morgan('Work') in Express

13. Commission & Payout Feature (Stretch Goal)

13.1 Commission Calculation Logic

Commission Calculation Logic

When `order.status` changes to 'Paid' via Stripe webhook:

For each item in `order.items`:

```
| itemRevenue   = item.price × item.qty |  
| platformFee   = itemRevenue × 0.10   (10%) |  
| vendorPayout  = itemRevenue - platformFee |
```

Store in order document:

```
{  
  items: [  
    {  
      productId:   '...',  
      vendorId:    '...',  
      price:       100,  
      qty:         2,  
      itemRevenue: 200,  
      platformFee: 20,  
      vendorPayout: 180  
    }  
  ]  
}
```

Vendor Dashboard shows per-sale breakdown:

```
| Sale Price:      Rs. 200 |  
| Platform Fee (10%): Rs. 20 |  
| Your Earnings:   Rs. 180 |
```

14. Environment Variables & Configuration

14.1 Backend .env

server/.env
— App —————
NODE_ENV=Developement
PORT=5000
STORE_MODE=multi # 'multi' 'single'
FRONTEND_URL=http://localhost:3000
— Database —————
MONGO_URI=mongodb+srv://<user>:<pass>@cluster0.mongodb.net/Final Project
— Auth —————
JWT_SECRET=your_super_secret_key_here_make_it_long
JWT_EXPIRES_IN=24h
— Cloudinary —————
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret
CLOUDINARY_UPLOAD_PRESET=Final Project_unsigned
— Stripe —————
STRIPE_SECRET_KEY=sk_test_...
STRIPE_WEBHOOK_SECRET=whsec_...
PLATFORM_COMMISSION_RATE=0.10

14.2 Frontend .env

client/.env
— All React env vars MUST start with VITE_ —————
VITE_API_URL=http://localhost:5000/api
VITE_CLOUDINARY_CLOUD_NAME=your_cloud_name
VITE_CLOUDINARY_UPLOAD_PRESET=Final Project_unsigned
VITE_STRIPE_PUBLISHABLE_KEY=pk_test_...
VITE_STORE_MODE=multi # must match backend STORE_MODE
VITE_STORE_NAME=Final Project # used in Navbar and page titles

15. Risks & Mitigations

Risk	Severity	Description	Mitigation
Stripe webhook not firing locally	MEDIUM	Stripe can't reach localhost	Use Stripe CLI: stripe listen --forward-to localhost:5000/api/stripe/webhook
Cloudinary limits exceeded	LOW	Free tier: 25 credits/month	Compress images before upload; limit to 3 images per product in MVP
MongoDB Atlas free tier (512MB)	LOW	Storage limit hit during demo	Use Atlas Data Explorer to monitor; delete test orders before demo
JWT token leakage via localStorage	MEDIUM	XSS attacks can steal token	Sanitize all user inputs; consider httpOnly cookie for production upgrade
Render cold start delays	MEDIUM	Free tier spins down after inactivity	Ping backend with uptime monitor; warn judges about first-load delay
CORS misconfiguration	HIGH	Frontend blocked from calling API	Set cors({ origin: FRONTEND_URL }) explicitly in Express; test early

16. Testing Checklist

16.1 Manual Testing Scenarios

#	Test Scenario	Expected Result	Test Data
1	Register as Customer	Redirect to home, JWT stored	test@email.com / Pass1234!
2	Register as Vendor (multi mode)	Vendor dashboard accessible	vendor@email.com / Pass1234!
3	Login with wrong password	401 error message displayed	bad password
4	Vendor adds product without image	Validation error shown	Omit image field
5	Customer adds product to cart	Cart icon badge increments	Any product page
6	Cart persists after page refresh	Items still in cart	Add items, refresh page
7	Complete Stripe test payment	Order created, status = Paid	Card: 4242 4242 4242 4242
8	Vendor sees new order in sales	Order appears in vendor dashboard	After step 7
9	Customer views order history	Order with status 'Paid' shown	My Orders page
10	Access vendor route as customer	403 Forbidden returned	Use customer JWT on /vendor/products
11	Vendor cannot edit another's product	403 returned	Use different vendor JWT
12	Admin deactivates a vendor	Vendor cannot login / products hidden	Admin panel user toggle

16.2 Stripe Test Cards Reference

Scenario	Card Number	Expected Outcome
Successful payment	4242 4242 4242 4242	Payment succeeds, order marked Paid
Payment declined	4000 0000 0000 0002	Stripe shows decline error
3D Secure authentication	4000 0025 0000 3155	3DS prompt shown, then success

Scenario	Card Number	Expected Outcome
Insufficient funds	4000 0000 0000 9995	Card declined with message

17. Glossary

Term	Definition
MERN	MongoDB, Express.js, React, Node.js,a full-stack JavaScript web Development stack
JWT	JSON Web Token,a compact, URL-safe token format for representing authentication claims
ODM	Object Document Mapper,Mongoose maps MongoDB documents to JavaScript objects
Multi-Vendor	A marketplace where multiple independent sellers list and sell their own products
Single-Vendor	An e-commerce store owned and operated by one seller/brand
Commission	A percentage of each sale taken by the platform operator as a service fee
Unsigned Preset	A Cloudinary configuration that allows direct browser uploads without exposing API secrets
Webhook	An HTTP callback sent by Stripe to notify the backend of payment events
Soft Delete	Marking a record as inactive (isActive: false) instead of physically removing it from DB
Redux Slice	A Redux Toolkit concept,combines reducers and actions for one feature into one file
PCI Compliance	Payment Card Industry standards for securely handling credit card data
DXA	Document eXtended Attribute unit in OOXML,1 inch = 1440 DXA
Middleware	Express.js functions that run between request receipt and route handler execution
STORE_MODE	Environment variable that toggles platform between 'single' and 'multi' vendor mode

18. Revision History

Version	Date	Author	Changes
1.0	19th February 2026	Ahmad Hassan	Initial PRD, Multi & Single-Vendor modes, full MERN stack specification

Document End

This PRD is approved for Development by Ahmad Hassan. All students should refer to this document throughout the Development. Questions or clarifications should be raised in the Web Ascend WhatsApp group before any architectural decisions are changed.